Technical University of Denmark

Written examination date: May 26, 2021

Course title: Computer Science Modelling

Course number: 02141

Aids allowed: All aids allowed for digital exams

Exam duration: 4 hours

Weighting: See the individual exercises

Grading: 7-step scale

Due to the Covid-19 situation the exam is conducted as a digital exam taken at home with the aids allowed for such exams. For this reason answers that can be obtained using a computer need to be properly explained and will not be considered acceptable otherwise. You must submit your answer as a single pdf-file; it is fine to integrate pictures of drawings into the pdf-file, but no attachments to the pdf-file will be considered.

If you think there are mistakes in the exam set (where in a written exam you would ask to be contacted by the teacher) please send an e-mail to fnie@dtu.dk for Formal Methods and to albl@dtu.dk for Regular Languages and Context Free Languages. Any announcements resulting from this will be communicated over the Microsoft Teams channel for the exam.

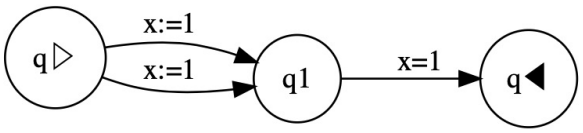
Answers Included

Exercises on Formal Methods

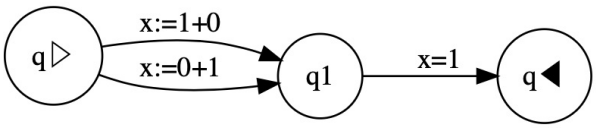
Exercise 1 (10%) Program Graphs

Consider the following program graphs:

Program graph A:



Program graph B:



Question 1a: *For each of program graphs A and B, does it satisfy the sufficient conditions of [FM Proposition 1.22] for being a deterministic system?*
(You should explain your answer.)

Solution (2%): No for both A and B because at $q_{\triangleright}$ there are two distinct edges emanating that do not have non-overlapping domains (they have the same non-empty domains).
Yes for A if you translate the graph into tuple notation and observe that the two distinct edges in the graph obtain the same representation and hence are the same.

Question 1b: *For each of program graphs A and B, does it satisfy the conditions of [FM Definition 1.21] for being a deterministic system?*
(You should explain your answer.)

Solution (2%): Yes for A and No for B. It is Yes for A because the two edges emanating from $q_{\triangleright}$ have the same actions. It is No for B because the two edges emanating from $q_{\triangleright}$ have different actions (even though they are semantically equivalent).

Question 1c: *For each of program graphs A and B, does it satisfy the sufficient conditions of [FM Proposition 1.25] for being an evolving system?*

(You should explain your answer.)

Solution (2%): No for both A and B because there is no edge emanating from q_1 that can be taken when the value of $\mathtt{x}$ is different from 1.

Question 1d: *For each of program graphs A and B, does it satisfy the conditions of [FM Definition 1.24] for being an evolving system?*
(You should explain your answer.)

Solution (2%): Yes for both A and B because all executions reaching q_1 will be with a memory where $\mathtt{x}$ is 1 and hence the emanating edge can be taken.

Question 1e: *For each of program graphs A and B, does there exist a program in Guarded Commands that generates the program graph according to [FM Section 2.2]?*
(You should explain your answer: in case of a positive answer show the program, in case of a negative answer argue why this cannot be the case.)

Solution (2%): No for both A and B. Inspection of [FM Section 2.2] shows that whenever we generate more than one edge that emanates from a node, then all these edges will be labelled with a boolean condition, which is not the case for neither programs A or B.

Exercise 2 (20%) Program Verification and Analysis

Consider program graph C from Figure 1 and consider the following execution sequence

$$\langle q_{\triangleright}; \sigma \rangle \overset{w}{\Longrightarrow}^* \langle q_{\blacktriangleleft}; \sigma' \rangle$$

where σ is given by $\sigma(\mathtt{x}) = 5$, $\sigma(\mathtt{y}) = 0$, $\sigma(\mathtt{z}) = 0$, $\sigma(\mathtt{zz}) = 0$.

Question 2a: *What is σ'?*
(You should explain your answer.)

Solution (4%): We get $\sigma'(\mathtt{x}) = 2$, $\sigma'(\mathtt{y}) = 3$, $\sigma'(\mathtt{z}) = 5$, $\sigma'(\mathtt{zz}) = 3$, and note that at q_5 the values of $\mathtt{y}$ and $\mathtt{z}$ will be two consecutive Fibonacci numbers (and the value of $\mathtt{y}$ will be equal to that of $\mathtt{zz}$),

Recall the Fibonacci numbers 1, 1, 2, 3, 5, 8 ... and let us write them as $fib(1) = 1$, $fib(2) = 1$, $fib(n+2) = fib(n) + fib(n+1)$ (for $n > 0$)

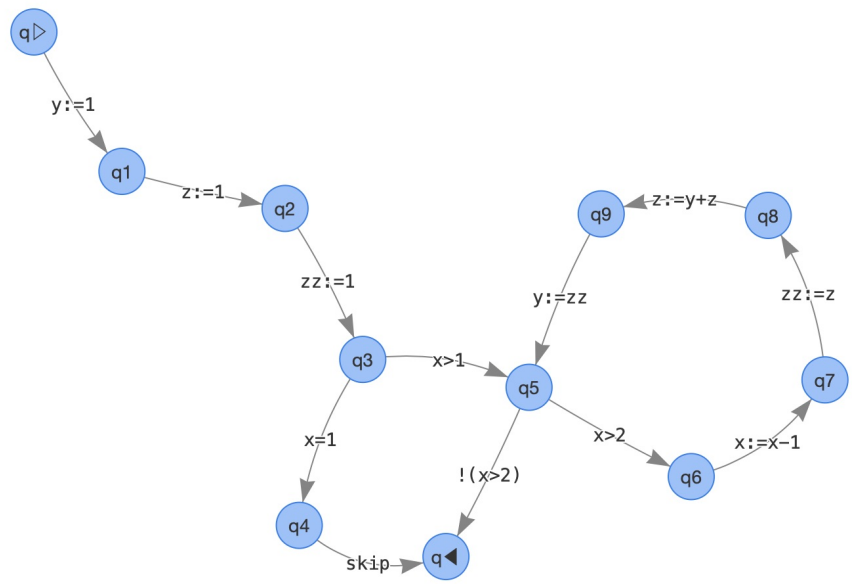


Figure 1: Program Graph C

.

Question 2b: *Complete the following partial predicate assignment*

$$\begin{array}{rcl} \mathbf{P}(q_{\triangleright}) & = & \mathtt{x} = \mathtt{n} \ \wedge \ \mathtt{n} > 0 \\ \mathbf{P}(q_5) & = & \cdots \\ \mathbf{P}(q_{\blacktriangleleft}) & = & \mathtt{z} = fib(\mathtt{n}) \end{array}$$

to a partially correct predicate assignment (in the sense of [FM, Section 3.3]), and list all the correct proof obligations that need to be checked.
(You do not have to check them in detail provided that they are correct.)

Solution (8%): A solution is

$$\begin{array}{rcl} \mathbf{P}(q_{\triangleright}) & = & \mathtt{x} = \mathtt{n} \ \wedge \ \mathtt{n} > 0 \\ \mathbf{P}(q_5) & = & \mathtt{n} - \mathtt{x} \geq 0 \ \wedge \ \mathtt{x} > 1 \ \wedge \ \mathtt{y} = fib(\mathtt{n} - \mathtt{x} + 1) \ \wedge \ \mathtt{z} = fib(\mathtt{n} - \mathtt{x} + 2) \\ \mathbf{P}(q_{\blacktriangleleft}) & = & \mathtt{z} = fib(\mathtt{n}) \end{array}$$

and note that at q_5 the value of $\mathtt{n} - \mathtt{x}$ gives the number of times the loop has been traversed.

We need to check the proof obligations:

$$\left[\begin{array}{l} \mathtt{x} = \mathtt{n} \wedge \\ \mathtt{n} > 0 \end{array}\right] \quad \begin{array}{l} \mathtt{y} := 1 \\ \mathtt{z} := 1 \\ \mathtt{zz} := 1 \\ \mathtt{x} > 1 \end{array} \quad \left[\begin{array}{l} \mathtt{n} - \mathtt{x} \geq 0 \wedge \\ \mathtt{x} > 1 \wedge \\ \mathtt{y} = fib(\mathtt{n} - \mathtt{x} + 1) \wedge \\ \mathtt{z} = fib(\mathtt{n} - \mathtt{x} + 2) \end{array}\right]$$

$$\left[\begin{array}{l} \mathtt{x} = \mathtt{n} \wedge \\ \mathtt{n} > 0 \end{array}\right] \quad \begin{array}{l} \mathtt{y} := 1 \\ \mathtt{z} := 1 \\ \mathtt{zz} := 1 \\ \mathtt{x} = 1 \end{array} \quad \left[\ \mathtt{z} = fib(\mathtt{n})\ \right]$$

$$\left[\begin{array}{l} \mathtt{n} - \mathtt{x} \geq 0 \wedge \\ \mathtt{x} > 1 \wedge \\ \mathtt{y} = fib(\mathtt{n} - \mathtt{x} + 1) \wedge \\ \mathtt{z} = fib(\mathtt{n} - \mathtt{x} + 2) \end{array}\right] \quad \begin{array}{l} \mathtt{skip} \\ \\ !(\mathtt{x} > 2) \end{array} \quad \left[\ \mathtt{z} = fib(\mathtt{n})\ \right]$$

$$\left[\begin{array}{l} \mathtt{n} - \mathtt{x} \geq 0 \wedge \\ \mathtt{x} > 1 \wedge \\ \mathtt{y} = fib(\mathtt{n} - \mathtt{x} + 1) \wedge \\ \mathtt{z} = fib(\mathtt{n} - \mathtt{x} + 2) \end{array}\right] \quad \begin{array}{l} \mathtt{x} > 2 \\ \mathtt{x} := \mathtt{x} - 1 \\ \mathtt{zz} := \mathtt{z} \\ \mathtt{z} := \mathtt{y} + \mathtt{z} \\ \mathtt{y} := \mathtt{zz} \end{array} \quad \left[\begin{array}{l} \mathtt{n} - \mathtt{x} \geq 0 \wedge \\ \mathtt{x} > 1 \wedge \\ \mathtt{y} = fib(\mathtt{n} - \mathtt{x} + 1) \wedge \\ \mathtt{z} = fib(\mathtt{n} - \mathtt{x} + 2) \end{array}\right]$$

We dispense with the detailed argument for why they are correct.

Next let us consider the detection of signs analysis as desribed in [FM, Chapter 4] and let us write an abstract state as (sx, sy, sz, szz) where sx is the sign of $\mathtt{x}$ etc. so that (0,+,-,0) represents an abstract state where $\mathtt{x}$ is 0, $\mathtt{y}$ is positive, $\mathtt{z}$ is negative and $\mathtt{zz}$ is 0.

Question 2c: *Construct the least analysis assignment* $\mathbf{A}$ *that has* $\mathbf{A}(q_{\triangleright}) = \{(+,+,+,+)\}$ *and give the results for* $\mathbf{A}(q_5), \mathbf{A}(q_6), \mathbf{A}(q_7), \mathbf{A}(q_{\blacktriangleleft})$.
(You should briefly explain how you obtain the answer.)

Solution (8%): Using the algorithm for chaotic iteration we get

$$\begin{array}{l} \mathbf{A}(q_{\triangleright}) = \mathbf{A}(q_1) = \mathbf{A}(q_2) = \mathbf{A}(q_3) = \mathbf{A}(q_4) = \mathbf{A}(q_6) = \{(+,+,+,+)\} \\ \mathbf{A}(q_5) = \mathbf{A}(q_7) = \mathbf{A}(q_8) = \mathbf{A}(q_9) = \mathbf{A}(q_{\blacktriangleleft}) = \\ \qquad \{(-,+,+,+),(0,+,+,+),(+,+,+,+)\} \end{array}$$

so in particular

$$\begin{array}{l} \mathbf{A}(q_5) = \{(-,+,+,+),(0,+,+,+),(+,+,+,+)\} \\ \mathbf{A}(q_6) = \{(+,+,+,+)\} \\ \mathbf{A}(q_7) = \{(-,+,+,+),(0,+,+,+),(+,+,+,+)\} \\ \mathbf{A}(q_{\blacktriangleleft})) = \{(-,+,+,+),(0,+,+,+),(+,+,+,+)\} \end{array}$$

We note that even though $\mathbf{A}(q_6) = \{(+,+,+,+)\}$ we do **not** get $\mathbf{A}(q_7) = \{(0,+,+,+),(+,+,+,+)\}$ because the analysis of [FM, Section 4.3] does not

treat subtraction by 1 any more precisely than subtraction by any other positive number.

Exercise 3 (10%) Information Flow

Consider the program graph C from Figure 1.

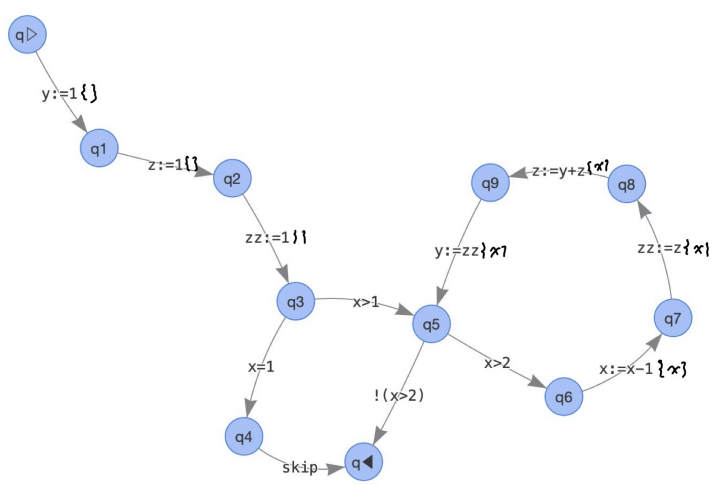
Question 3a: *Write a program in Guarded Commands for which the* **edges** *function of [FM, Section 2.2] generates program graph C.*

Solution (4%):

```
y:=1; z:=1; zz:=1;
if x=1 -> skip
[] x>1 ->
          do x>2 -> x:=x-1;
                    zz:=z;
                    z:=y+z;
                    y:=zz
          od
fi
```

Question 3b: *Draw the instrumented program graph [FM, Section 5.2] for the program constructed above (and that generates the program graph C).*

Solution (4%):



Question 34: *Would we obtain a secure program (in the sense that* **sec** *of [FM, Section 5.3] gives true) if we take* `x` *to be private and* `y`, `z`, *and* `zz` *to be public (using the confidentiality lattice [FM, Figure 5.7])?*
(You should explain your answer.)

Solution (2%): No, because the instrumented program graph shows an implicit flow from `x` to `y`, `z`, and `zz` which would not be permitted according to the confidentiality lattice [FM, Figure 5.7].

Exercise 4 (10%) Model Checking

Consider the program graph C from Figure 1 and the transition system constructed from it as explained in [FM, Section 6.4]. (Since the variables take values in the integers the transition system will have infinitely many states and therefore we shall not draw it.) But let us change the set of atomic propositions to include formulae of the form $x = n$ where x is a program variable (i.e. one of `x`, `y`, `z`, `zz`) and n is an integer. The label function we need is then defined by

$$\mathbf{L}\langle q; \sigma\rangle = \{\triangleright \mid q = q_{\triangleright}\} \cup \{\blacktriangleleft \mid q = q_{\blacktriangleleft}\} \cup \{x = n \mid \sigma(x) = n\}$$

Question 4a: *Does the formula* $\triangleright \wedge \ \mathtt{x} = 5 \Rightarrow \mathbf{EF}\,(\blacktriangleleft \wedge \mathtt{z} = 3)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): No, because the final result is `z`=5.

Question 4b: *Does the formula* $\triangleright \wedge \ \mathtt{x} = 5 \Rightarrow \mathbf{AF}\,(\blacktriangleleft \wedge \mathtt{z} = 3)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): No, because the final result is `z`=5.

Question 4c: *Does the formula* $\triangleright \wedge \ \mathtt{x} = 5 \Rightarrow \mathbf{EF}\,(\blacktriangleleft \wedge \mathtt{z} = 5)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): Yes, because the final result is indeed `z`=5.

Question 4d: *Does the formula* $\triangleright \wedge \ \mathtt{x} = 5 \Rightarrow \mathbf{AF}\,(\blacktriangleleft \wedge \mathtt{z} = 5)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): Yes, because the final result is indeed `z`=5, and the program graph is deterministic.

Question 4e: *Does the formula* $\triangleright \Rightarrow \mathbf{EF}\ \blacktriangleleft$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): No, the transition system will be stuck at $\langle q_3; \sigma\rangle$ when we have $\sigma(\mathtt{x}) < 1$.

Question 4f: *Does the formula* $\triangleright \Rightarrow \mathbf{AF}\ \blacktriangleleft$ *hold for the transition system?*

(You should explain your answer but do not have to be formal.)

Solution (1%): No, the transition system will be stuck at $\langle q_3; \sigma \rangle$ when we have $\sigma(\mathtt{x}) < 1$.

Question 4g: *Does the formula* $\triangleright \wedge\ \mathtt{x} = 5 \Rightarrow \mathbf{EF}\,(\mathtt{x} = 1)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): No, because a value of $\mathtt{x}$ higher than 1 will never be decremented to 1.

Question 4h: *Does the formula* $\triangleright \wedge\ \mathtt{x} = 5 \Rightarrow \mathbf{AF}\,(\mathtt{x} = 1)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): No, because a value of $\mathtt{x}$ higher than 1 will never be decremented to 1.

Question 4i: *Does the formula* $\triangleright \wedge\ \mathtt{x} = 5 \Rightarrow \mathbf{E}(\mathtt{x} = 5\ \mathbf{U}\ \mathtt{x} = 4)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): Yes, as witnessed by the path $q_\triangleright, q_1, q_2, q_3, q_5, q_6, q_7$.

Question 4j: *Does the formula* $\triangleright \wedge\ \mathtt{x} = 5 \Rightarrow \mathbf{A}(\mathtt{x} = 5\ \mathbf{U}\ \mathtt{x} = 4)$ *hold for the transition system?*
(You should explain your answer but do not have to be formal.)

Solution (1%): Yes, because the program graph is deterministic and we have the path $q_\triangleright, q_1, q_2, q_3, q_5, q_6, q_7$.

Exercises on Context-free Languages

Exercise 5 (25%)

In the lectures on program verification you were introduced to predicates. The following grammar provides a syntax for predicates similar to the one you have seen in the course:

$$
\begin{array}{lcl}
F & \rightarrow & \texttt{true} \mid F \texttt{ and } F \mid \texttt{not } F \mid \texttt{exists } V \texttt{ st } F \mid V \texttt{ equals } V \\
V & \rightarrow & \texttt{x} \mid V \texttt{ prime}
\end{array}
$$

where the set of non-terminal symbols is $N = \{F, V\}$, the set of terminal symbols is $T = \{\texttt{true}, \texttt{and}, \texttt{not}, \texttt{exists}, \texttt{st}, \texttt{equals}, \texttt{x}, \texttt{prime}\}$ and the initial symbol is F. Predicates are all strings accepted by the grammar.

(a) Design a set of datatypes that are suitable to store abstract representations (ASTs) of predicates and show how the formula

```
exists x prime st x equals x prime
```

would be represented as a value of your datatype.

Solution: In F# we could define the following types:

```
type F =
       | True
       | And of F * F
       | Not of F
       | Exists of V * F
       | Equals of V * V
and V =
       | X
       | Prime of V
```

The function signature would be represented by

```
Exists(Prime(X),Equals(X,Prime(X)))
```

(b) Consider the predicate composition operators `and`, `not` and `exist...st`. Fill the table below as follows. In the cell corresponding to row x and column y write YES if, according to the above grammar, the operators x and y can yield an ambiguity. Otherwise, write NO.

	not	and	exists...st
not			
and			
exists...st			

Solution:

	`not`	`and`	`exists...st`
`not`	NO	YES	NO
`and`	YES	YES	YES
`exists...st`	NO	YES	NO

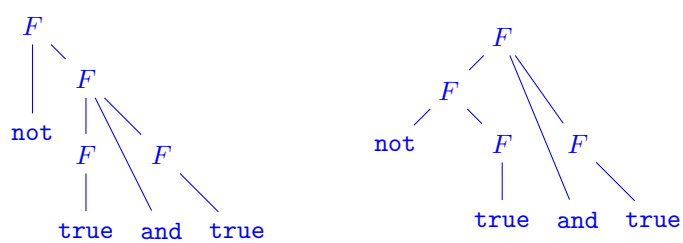
(c) For each of ambiguity identified in (b), showcase the ambiguity by providing an example of a formula that has at least two distinct parse trees.

Solution:

The ambiguity between `not` and `and` is due to undefined precedence and can be showcased by the formula

`not true and true`

and the two distinct parse trees:



The ambiguity between `not` and `exists...st` is due to undefined precedence and can be showcased similarly with the formula:

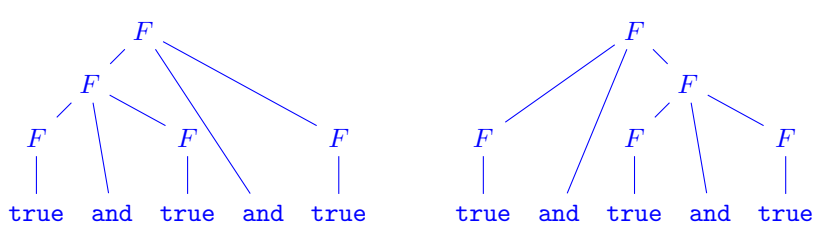
`exists x st true and true`

and the corresponding pair of parse trees (similar to the ones above).

The ambiguity between `and` and `and` is due to undefined associativity and can be showcased with the formula

`true and true and true`

and the two distinct parse trees



(d) Consider the PDA $(\{q\}, T, T \cup N, \delta, q, F, \{q\})$ where the transition function δ is defined as follows:

$$
\begin{array}{rcl}
\delta(q, \epsilon, F) & = & \{(q, F\texttt{and}F), (q, V\texttt{equals}V)\} \\
\delta(q, \texttt{true}, F) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{not}, F) & = & \{(q, F)\} \\
\delta(q, \texttt{exists}, F) & = & \{(q, V\texttt{st}F)\} \\
\delta(q, \epsilon, V) & = & \{(q, V\texttt{prime})\} \\
\delta(q, \texttt{x}, V) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{true}, \texttt{true}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{and}, \texttt{and}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{not}, \texttt{not}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{exists}, \texttt{exists}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{st}, \texttt{st}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{equals}, \texttt{equals}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{x}, \texttt{x}) & = & \{(q, \epsilon)\} \\
\delta(q, \texttt{prime}, \texttt{prime}) & = & \{(q, \epsilon)\}
\end{array}
$$

Check if the example formula of exercise (a) is accepted by the PDA. Do so by showing the accepting sequence of configurations (instantaneous descriptions) of the PDA starting with the initial one

$$(q, \texttt{exists x prime st x equals x prime}, F)$$

where q is the initial state of your PDA, `exists x prime st x equals x prime` is the input string and F is the initial stack symbol.

Solution:

```
  (q, exists x prime st x equals x prime, F)
⊢ (q,        x prime st x equals x prime, V st F)
⊢ (q,        x prime st x equals x prime, V prime st F)
⊢ (q,          prime st x equals x prime, prime st F)
⊢ (q,                st x equals x prime, st F)
⊢ (q,                   x equals x prime, F)
⊢ (q,                   x equals x prime, V equals V)
⊢ (q,                     equals x prime, equals V)
⊢ (q,                            x prime, V)
⊢ (q,                            x prime, V prime)
⊢ (q,                              prime, prime)
⊢ (q,                                   ϵ, ϵ)
```

(e) Does the PDA of (d) accept programs that are not accepted by the grammar in (a)? Explain your answer. You do not need to provide a formal answer (e.g. a formal proof); it suffices to provide the main intuitive insights of your argument.

Solution:

It should be easy to recognize that the PDA provided is very similar to the one that one would obtain from the standard construction of PDAs from CFGs. The only difference is that some predictions are "accelerated" (although the PDA is still non-deterministic).

For example, the transitions of the provided PDA for V are

$$\begin{array}{rcl} \delta(q, \epsilon, V) & = & \{(q, V\texttt{prime})\} \\ \delta(q, \texttt{x}, V) & = & \{(q, \epsilon)\} \end{array}$$

while with the standard construction we would obtain:

$$\delta'(q, \epsilon, V) \quad = \quad \{(q, V\texttt{prime}), (q, \texttt{x})\}$$

It is easy to see that accelerated transitions of δ as in

$$\begin{array}{rl} & (\text{q, } \texttt{x}\text{w, Vs}) \\ \vdash & (\text{q, w, s}) \end{array}$$

can be mimicked by δ' with

$$\begin{array}{rl} & (\text{q, } \texttt{x}\text{w, Vs}) \\ \vdash & (\text{q, } \texttt{x}\text{w, } \texttt{x}\text{s}) \\ \vdash & (\text{q, w, s}) \end{array}$$

The same arguments apply to F.

A formal proof (not requested by the exercise) can be constructed on this main insight.

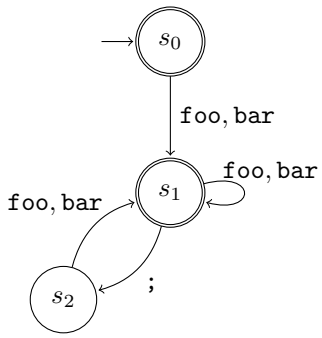
The conclusion is that all words accepted by the PDA are accepted by the grammar.

Exercises on Regular Languages

Exercise 6 (25%)

In this exercise we consider a simple programming language, in which two functions (`foo` and `bar`) that can be composed or invoked in sequence. We will consider the following three proposals for the syntax of the programming language:

(1) The automaton A



(2) The regular expression B

$$(\texttt{foo;} + \texttt{bar;})^*$$

(3) The grammar C

$$\begin{array}{lcl} S & \rightarrow & F \mid S\texttt{;}F \\ F & \rightarrow & \texttt{foo} \mid \texttt{bar} \end{array}$$

where the set $\{\texttt{foo}, \texttt{bar}, \texttt{;}\}$ acts as input alphabet for A and B, and as the set of terminal symbols for C, and where the non-terminal symbols in C are S (initial) and F.

(a) Consider the programs accepted by A, B and C, i.e. their languages $\mathcal{L}(\mathsf{A})$, $\mathcal{L}(\mathsf{B})$, $\mathcal{L}(\mathsf{C})$. Explain, for each pair of languages, their relation: Does one contain the other? Do they overlap? Answer by filling the below tables and explain your answers. You do not need to explain the (trivial) pre-filled cases.

More precisely, in the below table write YES in the cell corresponding to row x and column y if the language x is contained in the language y. Otherwise, write NO.

$\mathcal{L}(\dots) \subseteq \mathcal{L}(\dots)$	$\mathcal{L}(A)$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	YES		
$\mathcal{L}(\mathsf{B})$		YES	
$\mathcal{L}(\mathsf{C})$			YES

Moreover, in the table below write YES in the cell corresponding to row x and column y if the language x overlaps with language y (i.e. they have at least one word/program in common). Otherwise, write NO.

$(\mathcal{L}(\dots) \cap \mathcal{L}(\dots)) \neq \emptyset$	$\mathcal{L}(\mathsf{A})$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	YES		
$\mathcal{L}(\mathsf{B})$		YES	
$\mathcal{L}(\mathsf{C})$			YES

Solution:

$\mathcal{L}(\dots) \subseteq \mathcal{L}(\dots)$	$\mathcal{L}(A)$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	YES	NO	NO
$\mathcal{L}(\mathsf{B})$	NO	YES	NO
$\mathcal{L}(\mathsf{C})$	YES	NO	YES

$(\mathcal{L}(\dots) \cap \mathcal{L}(\dots)) \neq \emptyset$	$\mathcal{L}(\mathsf{A})$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	YES	YES	YES
$\mathcal{L}(\mathsf{B})$	YES	YES	NO
$\mathcal{L}(\mathsf{C})$	YES	NO	YES

A vs B: the languages overlap but neither of them contains the other. This can be shown with three programs:

- ϵ is accepted by both A and B;
- `foo` is accepted by A but not by B;
- `foo;` is accepted by B but not by A.

A vs C: the language of C is strictly contained in the language of A. To see that the language of A is not contained in C we just observe that the empty string is accepted by A but not by C.

To see that the language of C is contained in the language of A we can provide a proof by induction. We need to show that for every string w generated by S in grammar C, w is accepted by A.

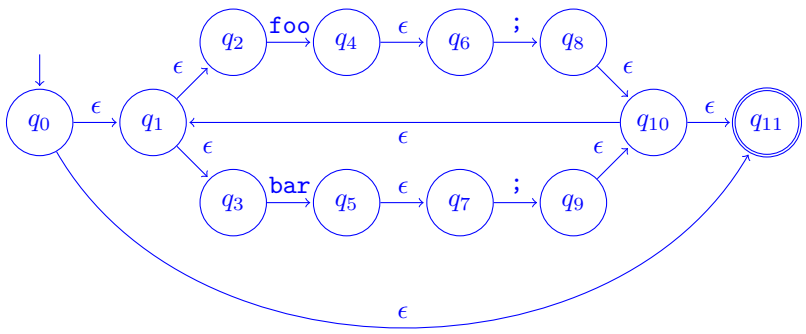
Base cases: The symbol S of grammar C generates strings `foo` and `bar` with production $S \to F$, which are clearly accepted by A.

Inductive step: Assume that w is derived as follows $S \Rightarrow S\texttt{;}F \Rightarrow^* w$. Then $w = w'\texttt{;}x$ with $x \in \{\texttt{foo}, \texttt{bar}\}$. The induction hypothesis is that w' is accepted by A. Hence, after reading w' the automaton is in state s_1 (it cannot be in s_0 since w cannot be the empty string). Reading input symbol `;` leads the automaton to state s_2 and reading then input symbol x (for any of the two options) leads the automaton back to the accepting state s_1. Hence, w is accepted by A.

B vs C: the languages do not overlap. B accepts the empty string but C does not. All other strings accepted by B finish with `;`, but no such word can be accepted by C. To see this we observe that the only occurrence of `;` in the body of a production is followed by F, which cannot generate the empty string.

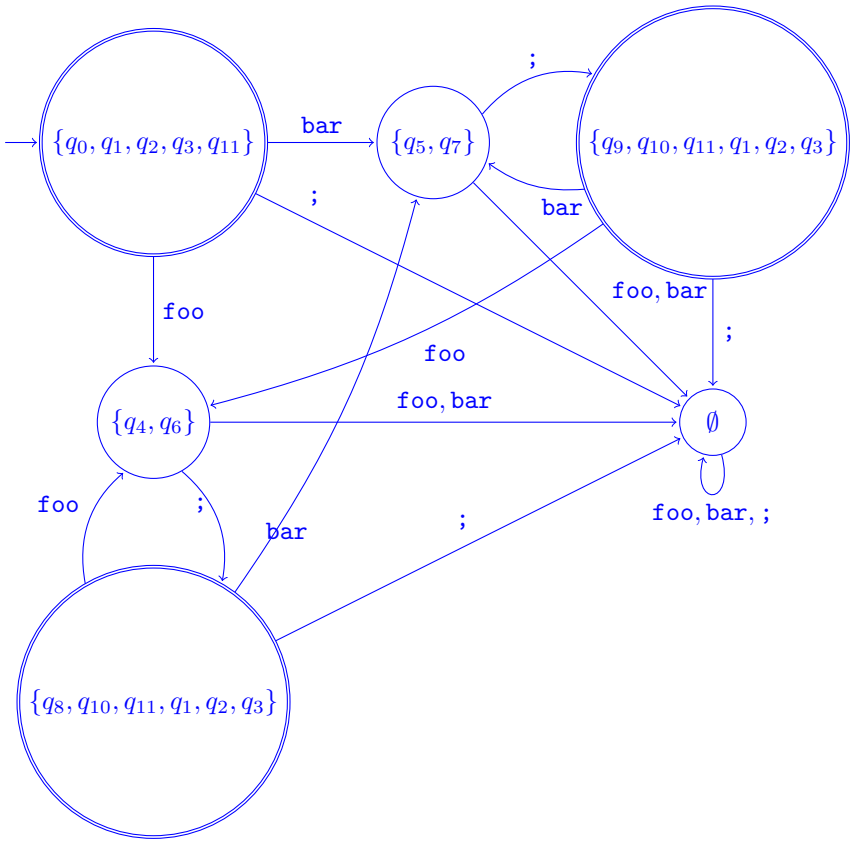
(b) Convert the regular expression B into an ϵ-NFA using the basic algorithm for translating a regular expression into an ϵ-NFA seen in the course. Provide the result as a transition diagram and do not apply any optimisation (i.e. do not remove ϵ-transitions).

Solution:



(c) Convert the ϵ-NFA obtained in (b) into a DFA using the procedure seen in the course. Provide the result as a transition diagram. Do not include unreachable states. Do include the state $\emptyset$ if it is reachable.

Solution:

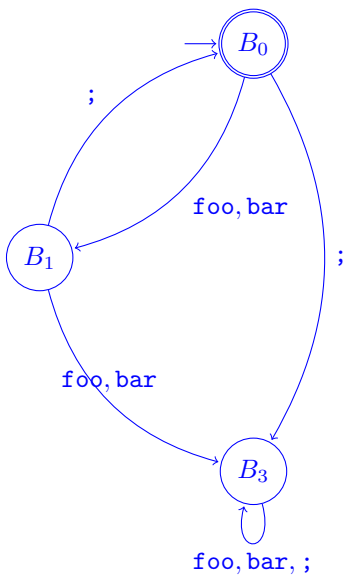


(d) Is the DFA obtained in (c) minimal or can you find an equivalent one that

is smaller? Explain your answer.

Solution:

One solution is to apply the partition refinement algorithm seen in class. This will yield the following minimized DFA:



where

- $B_0 = \{\{q_0, q_1, q_2, q_3, q_{11}\}, \{q_8, q_{10}, q_{11}, q_1, q_2, q_3\}, \{q_9, q_{10}, q_{11}, q_1, q_2, q_3\}\}$
- $B_1 = \{\{q_4, q_6\}, \{q_5, q_7\}\}$
- $B_3 = \{\emptyset\}$

Correctness of the solution (equivalence to the departing automata) follows from the correctness of the algorithm.